Why Program?

Computer – programmable machine designed to follow instructions

Program – instructions in computer memory to make it do something

Programmer – person who writes instructions (programs) to make computer perform a task

SO, without programmers, no programs; without programs, a computer cannot do anything

Main Hardware Component Categories:

1. Central Processing Unit (CPU)
2. Main Memory
3. Secondary Memory / Storage
4. Input Devices
5. Output Devices

Central Processing Unit (CPU)

Comprised of:

- Control Unit
  - Retrieves and decodes program instructions
  - Coordinates activities of all other parts of computer
- Arithmetic & Logic Unit
  - Hardware optimized for high-speed numeric calculation
  - Hardware designed for true/false, yes/no decisions

CPU Organization

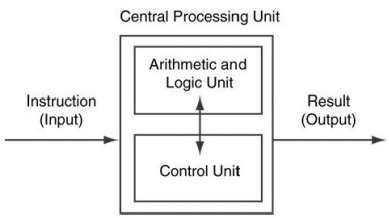


Figure 1-3

Main Memory

- It is volatile. Main memory is erased when program terminates or computer is turned off
- Also called Random Access Memory (RAM)
- Organized as follows:
  - bit: smallest piece of memory. Has values 0 (off, false) or 1 (on, true)
  - byte: 8 consecutive bits. Bytes have addresses.

Main Memory

- Addresses – Each byte in memory is identified by a unique number known as an *address*.

Secondary Storage

- Non-volatile: data retained when program is not running or computer is turned off
- Comes in a variety of media:
  - magnetic: traditional hard drives that use a moveable mechanical arm to read/write
  - solid-state: data stored in chips, no moving parts
  - optical: CD-ROM, DVD
  - Flash drives, connected to the USB port

Input Devices

- Devices that send information to the computer from outside
- Many devices can provide input:
  - Keyboard, mouse, touchscreen, scanner, digital camera, microphone
  - Disk drives, CD drives, and DVD drives

Software-Programs That Run on a Computer

- Categories of software:
  - System software: programs that manage the computer hardware and the programs that run on them.
    - *Examples*: operating systems, utility programs, software development tools
  - Application software: programs that provide services to the user.
    - *Examples* : word processing, games, programs to solve specific problems

Programs and Programming Languages

- A program is a set of instructions that the computer follows to perform a task
- We start with an *algorithm*, which is a set of well-defined steps.

Example Algorithm for Calculating Gross Pay

1. Display a message on the screen asking "How many hours did you work?"
2. Wait for the user to enter the number of hours worked. Once the user enters a number, store it in memory.
3. Display a message on the screen asking "How much do you get paid per hour?"
4. Wait for the user to enter an hourly pay rate. Once the user enters a number, store it in memory.
5. Multiply the number of hours by the amount paid per hour, and store the result in memory.
6. Display a message on the screen that tells the amount of money earned. The message must include the result of the calculation performed in Step 5.

Machine Language

- Although the previous algorithm defines the steps for calculating the gross pay, it is not ready to be executed on the computer.
- The computer only executes *machine language* instructions

Machine Language

- Machine language instructions are binary numbers, such as

  1011010000000101

- Rather than writing programs in machine language, programmers use *programming languages*.

Programs and Programming Languages

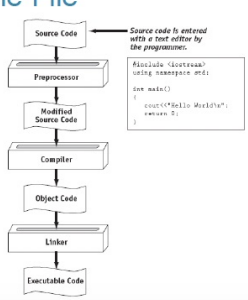
- Types of languages:
  - Low-level: used for communication with computer hardware directly. Often written in binary machine code (0's/1's) directly.
  - High-level: closer to human language

From a High-Level Program to an Executable File

a) Create file containing the program with a text editor.
b) Run preprocessor to convert source file directives to source code program statements.
c) Run compiler to convert source program into machine instructions.
d) Run linker to connect hardware-specific code to machine instructions, producing an executable file.

- Steps b–d are often performed by a single command or button click.
- Errors detected at any step will prevent execution of following steps.

From a High-Level Program to an Executable File



Integrated Development Environments (IDEs)

- An integrated development environment, or IDE, combine all the tools needed to write, compile, and debug a program into a single software application.
- Examples are Microsoft Visual Studio, Turbo C++ Explorer, Red Panda C++, etc.

What is a Program Made of?

- Common elements in programming languages:
  - Key Words
  - Programmer-Defined Identifiers
  - Operators
  - Punctuation
  - Syntax

Key Words

- Also known as reserved words
- Have a special meaning in C++
- Can not be used for any other purpose
- Key words in the Program 1-1: `using`, `namespace`, `int`, `double`, and `return`

Programmer-Defined Identifiers

- Names made up by the programmer
- Not part of the C++ language
- Used to represent various things: variables (memory locations), functions, etc.
- In Program 1-1: `hours`, `rate`, and `pay`.

Operators

- Used to perform operations on data
- Many types of operators:
  - Arithmetic - ex: `+`, `-`, `*`, `/`
  - Assignment – ex: `=`
- Some operators in Program1-1:
  `<<` `>>` `=` `*`

Punctuation

- Characters that mark the end of a statement, or that separate items in a list
- In Program 1-1: `,` and `;`

Syntax

- The rules of grammar that must be followed when writing a program
- Controls the use of key words, operators, programmer-defined symbols, and punctuation

Variables

- A variable is a named storage location in the computer's memory for holding a piece of data.
- In Program 1-1 we used three variables:
  - The `hours` variable was used to hold the hours worked
  - The `rate` variable was used to hold the pay rate
  - The `pay` variable was used to hold the gross pay

Variable Definitions

- To create a variable in a program you must write a variable definition (also called a variable declaration)
- Here is the statement from Program 1-1 that defines the variables:

```
double hours, rate, pay;
```

Variable Definitions

- There are many different types of data, which you will learn about in this course.
- A variable holds a specific type of data.
- The variable definition specifies the type of data a variable can hold, and the variable name.

Variable Definitions

- Once again, line 7 from Program 1-1:

```
double hours, rate, pay;
```

- The word **`double`** specifies that the variables can hold double-precision floating point numbers. (You will learn more about that in Chapter 2)

Input, Processing, and Output

Three steps that a program typically performs:

1) **Gather input data:**
   - from keyboard
   - from files on disk drives
2) **Process the input data**
3) **Display the results as output:**
   - send it to the screen
   - write to a file

The Programming Process

1. Clearly define what the program is to do.
2. Visualize the program running on the computer.
3. Use design tools such as a hierarchy chart, flowcharts, or pseudocode to create a model of the program.
4. Check the model for logical errors.
5. Type the code, save it, and compile it.
6. Correct any errors found during compilation. Repeat Steps 5 and 6 as many times as necessary.
7. Run the program with test data for input.
8. Correct any errors found while running the program. Repeat Steps 5 through 8 as many times as necessary.
9. Validate the results of the program.

Define what the program is to do

- This step requires that you identify the purpose of the program, the information that is to be input, the processing that is to take place, and the desired output.

Define what the program is to do

- Purpose: to calculate the user's gross pay.
- Input: number of hours worked, hourly pay rate.
- Process: multiply number of hours worked by hourly pay rate. The result is the user's gross pay.
- Output: display a message indicating the user's gross pay.

Visualize the program

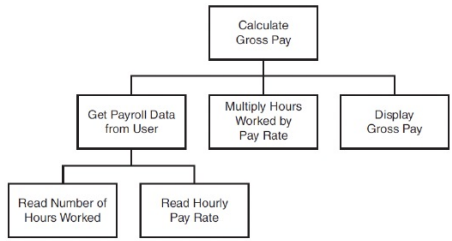
- Try to imagine what the computer screen looks like while the program is running.
- If it helps, draw pictures of the screen, with sample input and output, at various points in the program.

```
How many hours did you work? 10
How much do you get paid per hour? 15
You have earned $150
```

Use design tools to create a model of the program

- Three common design tools are hierarchy charts, flowcharts, and pseudo-code.
- A hierarchy chart is a diagram that graphically depicts the structure of a program. It has boxes that represent each step in the program. The boxes are connected in a way that illustrates their relationship to one another.

Hierarchy chart



Flowchart

- A flowchart is a diagram that shows the logical flow of a program.
- It is a useful tool for planning each operation a program performs and the order in which the operations are to occur.

Pseudocode

- Pseudocode is a cross between human language and a programming language.
- Although the computer can't understand pseudocode, programmers often find it helpful to write an algorithm in a language that's "almost" a programming language, but still very similar to natural language.

Pseudocode

Pseudocode that describes the pay-calculating program

Get payroll data.
Calculate gross pay.
Display gross pay.

Pseudocode

Display "How many hours did you work?".
Input hours.
Display "How much do you get paid per hour?".
Input rate.
Store the value of hours times rate in the pay variable.
Display the value in the pay variable.

Procedural and Object-Oriented Programming （OOP）

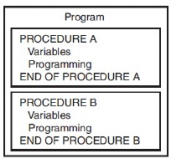
- Procedural programming: focus is on the process. Procedures/functions are written to process data.
- Object-Oriented programming: focus is on objects, which contain data and the means to manipulate the data. Messages sent to objects to perform operations.

Procedural Programming

- In procedural programming, the programmer constructs procedures (or functions, as they are called in C++).
- The procedures are collections of programming statements that perform a specific task. The procedures each contain their own variables and commonly share variables with other procedures.

Procedural Programming

- Different procedures (or functions) in a C++ program



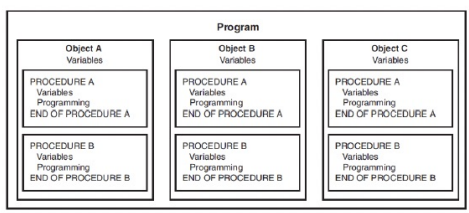
Object-Oriented Programming

- Object-oriented programming (OOP), is centered on the object.
- An object is a programming element that contains data and the procedures that operate on the data. It is a self-contained unit.

Object-Oriented Programming

- The objects contain, within themselves, both information and the ability to manipulate the information.
- Operations are carried out on the information in an object by sending the object a message. When an object receives a message instructing it to perform some operation, it carries out the instruction.

Objects in a C++ program



Special Characters

Character	Name	Meaning
//	Double slash	Beginning of a comment
#	Pound sign	Beginning of preprocessor directive
< >	Open/close brackets	Enclose filename in #include
( )	Open/close parentheses	Used when naming a function
{ }	Open/close brace	Encloses a group of statements
" "	Open/close quotation marks	Encloses string of characters
;	Semicolon	End of a programming statement

The `\n` Escape Sequence

- You can also use the `\n` escape sequence to start a new line of output. This will produce two lines of output:

```
cout << "Programming is\n";
cout << "fun!";
```

Notice that the `\n` is INSIDE the string.

The `#include` Directive

- Inserts the contents of another file into the program
- This is a preprocessor directive, not part of C++ language
- `#include` lines not seen by compiler
- Do not place a semicolon at end of `#include` line

Variables and Literals

- Variable: a storage location in memory
  - Has a name and a type of data it can hold
  - Must be defined before it can be used:

```
int item;
```

Literals

- Literal: a value that is written into a program's code.

```
"hello, there" (string literal)
12 (integer literal)
```

Identifiers

- An identifier is a programmer-defined name for some part of a program: variables, functions, etc.

C++ Key Words

Table 2-4 The C++ Key Words

alignas	const	for	private	throw
alignof	constexpr	friend	protected	true
and	const_cast	goto	public	try
and_eq	continue	if	register	typedef
asm	decltype	inline	reinterpret_cast	typeid
auto	default	int	return	typename
bitand	delete	long	short	union
bitor	do	mutable	signed	unsigned
bool	double	namespace	sizeof	using
break	dynamic_cast	new	static	virtual
case	else	noexcept	static_assert	void
catch	enum	not	static_cast	volatile
char	explicit	not_eq	struct	wchar_t
char16_t	export	nullptr	switch	while
char32_t	extern	operator	template	xor
class	false	or	this	xor_eq
compl	float	or_eq	thread_local	

You cannot use any of the C++ key words as an identifier. These words have reserved meaning.

Variable Names

- A variable name should represent the purpose of the variable. For example:

```
itemsOrdered
```

The purpose of this variable is to hold the number of items ordered.

Identifier Rules

- The first character of an identifier must be an alphabetic character or and underscore (_),
- After the first character you may use alphabetic characters, numbers, or underscore characters.
- Upper- and lowercase characters are distinct

Valid and Invalid Identifiers

IDENTIFIER	VALID?	REASON IF INVALID
`totalSales`	Yes	
`total_Sales`	Yes	
`total.Sales`	No	Cannot contain `.`
`4thQtrSales`	No	Cannot begin with digit
`totalSale$`	No	Cannot contain `$`

Integer Data Types

- Integer variables can hold whole numbers such as 12, 7, and -99.

Table 2-6 Integer Data Types

Data Type	Typical Size	Typical Range
short int	2 bytes	−32,768 to +32,767
unsigned short int	2 bytes	0 to +65,535
int	4 bytes	−2,147,483,648 to +2,147,483,647
unsigned int	4 bytes	0 to 4,294,967,295
long int	4 bytes	−2,147,483,648 to +2,147,483,647
unsigned long int	4 bytes	0 to 4,294,967,295
long long int	8 bytes	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned long long int	8 bytes	0 to 18,446,744,073,709,551,615

Defining Variables

- Variables of the same type can be defined
  - On separate lines:

```
int length;
int width;
unsigned int area;
```

  - On the same line:

```
int length, width;
unsigned int area;
```

- Variables of different types must be in different definitions

Integer Literals

- Integer literals are stored in memory as `int`s by default
- To store an integer constant in a long memory location, put '`L`' at the end of the number: `1234L`
- To store an integer constant in a long long memory location, put '`LL`' at the end of the number: `324LL`
- Constants that begin with '`0`' (zero) are base 8: `075`
- Constants that begin with '`0x`' are base 16: `0x75A`

The `char` Data Type

- Used to hold characters or very small integer values
- Usually 1 byte of memory
- Numeric value of character from the character set is stored in memory:

CODE:
```
char letter;
letter = 'C';
```

MEMORY:
letter

67

ASCII characters

https://www.ibm.com/docs/en/sdse/6.4.0?topic=configuration-ascii-characters-from-33-126

ASCII code	Character	ASCII code	Character	ASCII code	Character
33	! exclamation point	34	" double quotation	35	# number sign
36	$ dollar sign	37	% percent sign	38	& ampersand
39	' apostrophe	40	(left parenthesis	41	) right parenthesis
42	* asterisk	43	+ plus sign	44	, comma
45	- hyphen	46	. period	47	/ slash
48	0	49	1	50	2
51	3	52	4	53	5
54	6	55	7	56	8
57	9	58	: colon	59	; semicolon
60	< less-than sign	61	= equals sign	62	> greater-than sign
63	? question mark	64	@ at sign	65	A uppercase a
66	B uppercase b	67	C uppercase c	68	D uppercase d
69	E uppercase e	70	F uppercase f	71	G uppercase g

Character Strings

- A series of characters in consecutive memory locations:
  `"Hello"`
- Stored with the null terminator, `\0`, at the end:
- Comprised of the characters between the `"  "`

H	e	l	l	o	\0

The C++ `string` Class

- Special data type supports working with strings
  `#include <string>`
- Can define `string` variables in programs:
  `string firstName, lastName;`
- Can receive values with assignment operator:
  `firstName = "George";`
  `lastName = "Washington";`
- Can be displayed via `cout`
  `cout << firstName << " " << lastName;`

Floating-Point Data Types

Table 2-8 Floating Point Data Types on PCs

Data Type	Key Word	Description
Single precision	`float`	4 bytes. Numbers between ±3.4E-38 and ±3.4E38
Double precision	`double`	8 bytes. Numbers between ±1.7E-308 and ±1.7E308
Long double precision	`long double*`	8 bytes. Numbers between ±1.7E-308 and ±1.7E308

Floating-Point Literals

- Can be represented in
  - Fixed point (decimal) notation:
    `31.4159` `0.0000625`
  - E notation:
    `3.14159E1` `6.25e-5`
- Are `double` by default
- Can be forced to be float (`3.14159f`) or long double (`0.0000625L`)

The `bool` Data Type

- Represents values that are `true` or `false`
- `bool` variables are stored as small integers
- `false` is represented by 0, `true` by 1:

```
bool allDone = true;
bool finished = false;
```

allDone	finished
1	0

Determining the Size of a Data Type

- The `sizeof` operator gives the size of any data type or variable:

```
double amount;
cout << "A double is stored in "
     << sizeof(double) << "bytes\n";
cout << "Variable amount is stored in "
     << sizeof(amount)
     << "bytes\n";
```

Declaring Variables With the `auto` Key Word

- C++ 11 introduces an alternative way to define variables, using the `auto` key word and an initialization value. Here is an example:

```
auto amount = 100;   <-- int
```

- The `auto` key word tells the compiler to determine the variable's data type from the initialization value.

```
auto interestRate= 12.0;   <-- double
auto stockCode = 'D';      <-- char
auto customerNum = 459L;   <-- long
```

Scope

- The scope of a variable: the part of the program in which the variable can be accessed
- A variable cannot be used before it is defined

Arithmetic Operators

- Used for performing numeric calculations
- C++ has unary, binary, and ternary operators:
  - unary (1 operand) `-5`
  - binary (2 operands) `13 - 7`
  - ternary (3 operands) `exp1 ? exp2 : exp3`

Binary Arithmetic Operators

SYMBOL	OPERATION	EXAMPLE	VALUE OF ans
+	addition	`ans = 7 + 3;`	10
-	subtraction	`ans = 7 - 3;`	4
*	multiplication	`ans = 7 * 3;`	21
/	division	`ans = 7 / 3;`	2
%	modulus	`ans = 7 % 3;`	1

A Closer Look at the / Operator

- / (division) operator performs integer division if both operands are integers

```
cout << 13 / 5;     // displays 2
cout << 91 / 7;     // displays 13
```

- If either operand is floating point, the result is floating point

```
cout << 13 / 5.0;   // displays 2.6
cout << 91.0 / 7;   // displays 13.0
```

Comments

- Used to document parts of the program
- Intended for persons reading the source code of the program:
  - Indicate the purpose of the program
  - Describe the use of variables
  - Explain complex sections of code
- Are ignored by the compiler

Named Constants

- Named constant (constant variable): variable whose content cannot be changed during program execution
- Used for representing constant values with descriptive names:

```
const double TAX_RATE = 0.0675;
const int NUM_STATES = 50;
```

- Often named in uppercase letters

Programming Style

- The visual organization of the source code
- Includes the use of spaces, tabs, and blank lines
- Does not affect the syntax of the program
- Affects the readability of the source code

Displaying a Prompt

- A prompt is a message that instructs the user to enter data.
- You should always use **`cout`** to display a prompt before each **`cin`** statement.

```
cout << "How tall is the room? ";
cin >> height;
```

The `cin` Object

- Can be used to input more than one value:

```
cin >> height >> width;
```

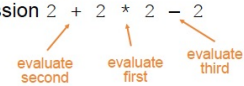
- Multiple values from keyboard must be separated by spaces
- Order is important: first value entered goes to first variable, etc.

Order of Operations

In an expression with more than one operator, evaluate in this order:

- `-` (unary negation), in order, left to right
- `*` `/` `%`, in order, left to right
- `+` `-`, in order, left to right

In the expression `2 + 2 * 2 - 2`



Hierarchy of Types

Highest: `long double`
`double`
`float`
`unsigned long`
`long`
`unsigned int`
Lowest: `int`

Ranked by largest number they can hold

Type Coercion

- Type Coercion: automatic conversion of an operand to another data type
- Promotion: convert to a higher type
- Demotion: convert to a lower type

Coercion Rules

1) `char`, `short`, `unsigned short` automatically promoted to `int`
2) When operating on values of different data types, the lower one is promoted to the type of the higher one.
3) When using the = operator, the type of expression on right will be converted to type of variable on left

Overflow and Underflow

- Occurs when assigning a value that is too large (overflow) or too small (underflow) to be held in a variable
- Variable contains value that is 'wrapped around' set of possible values
- Different systems may display a warning/error message, stop the program, or continue execution using the incorrect value

Overflow and Underflow

Program 3-7

```
// This program demonstrates integer overflow and underflow.
#include <iostream>
using namespace std;

int main()
{
     // testVar is initialized with the maximum value for a short.
     short testVar = 32767;

     // Display testVar.
     cout << testVar << endl;

     // Add 1 to testVar to make it overflow.
     testVar = testVar + 1;
     cout << testVar << endl;

     // Subtract 1 from testVar to make it underflow.
     testVar = testVar - 1;
     cout << testVar << endl;
     return 0;
}
```

Program Output

```
32767
-32768
32767
```

Type Casting

- Used for manual data type conversion
- Useful for floating point division using ints:

```
double m;
m = static_cast<double>(y2-y1)
                        /(x2-x1);
```

- Useful to see `int` value of a `char` variable:

```
char ch = 'C';
cout << ch << " is "
     << static_cast<int>(ch);
```

C-Style and Prestandard Type Cast Expressions

- C-Style cast: data type name in `()`

```
cout << ch << " is " << (int)ch;
```

- Prestandard C++ cast: value in `()`

```
cout << ch << " is " << int(ch);
```

- Both are still supported in C++, although `static_cast` is preferred

Multiple Assignment and Combined Assignment

- The = can be used to assign a value to multiple variables:

```
x = y = z = 5;
```

- Value of = is the value that is assigned
- Associates right to left:

```
x = (y = (z = 5));
```

value is 5 — value is 5 — value is 5

Formatting Output

- Can control how output displays for numeric, string data:
  - size
  - position
  - number of digits
- Requires `iomanip` header file

Stream Manipulators

- Used to control how an output field is displayed
- Some affect just the next value displayed:
  - `setw(x)`: print in a field at least `x` spaces wide. Use more spaces if field is not wide enough

Stream Manipulators

- Some affect values until changed again:
  - `fixed`: use decimal notation for floating-point values
  - `setprecision(x)`: when used with `fixed`, print floating-point value using `x` digits after the decimal. Without `fixed`, print floating-point value using x significant digits
  - `showpoint`: always print decimal for floating-point values

More Stream Manipulators in Program 3-17

Program 3-17

```
// This program asks for sales amounts for 3 days. The total
// sales are calculated and displayed in a table.
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
     double day1, day2, day3, total;

     // Get the sales for each day.
     cout << "Enter the sales for day 1: ";
     cin >> day1;
     cout << "Enter the sales for day 2: ";
     cin >> day2;
     cout << "Enter the sales for day 3: ";
     cin >> day3;

     // Calculate the total sales.
     total = day1 + day2 + day3;

```

Continued…

More Stream Manipulators in Program 3-17

```
22      // Display the sales amounts.
23      cout << "\nSales Amounts\n";
24      cout << "-------------\n";
25      cout << setprecision(2) << fixed;
26      cout << "Day 1: " << setw(8) << day1 << endl;
27      cout << "Day 2: " << setw(8) << day2 << endl;
28      cout << "Day 3: " << setw(8) << day3 << endl;
29      cout << "Total: " << setw(8) << total << endl;
30      return 0;
31  }
```

Program Output with Example Input Shown in Bold

```
Enter the sales for day 1: 1321.87 Enter
Enter the sales for day 2: 1869.26 Enter
Enter the sales for day 3: 1403.77 Enter

Sales Amounts
-------------
Day 1:     1321.87
Day 2:     1869.26
Day 3:     1403.77
Total:     4594.90
```

Stream Manipulators

Table 3-12

Stream Manipulator	Description
`setw(n)`	Establishes a print field of *n* spaces.
`fixed`	Displays floating-point numbers in fixed point notation.
`showpoint`	Causes a decimal point and trailing zeroes to be displayed, even if there is no fractional part.
`setprecision(n)`	Sets the precision of floating-point numbers.
`left`	Causes subsequent output to be left justified.
`right`	Causes subsequent output to be right justified.

Working with Characters and `string` Objects

- Using `cin` with the >> operator to input strings can cause problems:
- It passes over and ignores any leading *whitespace characters (spaces, tabs, or line breaks)*
- To work around this problem, you can use a C++ function named `getline`.

Using `getline` in Program 3-19

Program 3-19

```
1   // This program demonstrates using the getline function
2   // to read character data into a string object.
3   #include <iostream>
4   #include <string>
5   using namespace std;
6
7   int main()
8   {
9      string name;
10     string city;
11
12     cout << "Please enter your name: ";
13     getline(cin, name);
14     cout << "Enter the city you live in: ";
15     getline(cin, city);
16
17     cout << "Hello, " << name << endl;
18     cout << "You live in " << city << endl;
19     return 0;
20  }
```

Program Output with Example Input Shown in Bold

```
Please enter your name: Kate Smith [Enter]
Enter the city you live in: Raleigh [Enter]
Hello, Kate Smith
You live in Raleigh
```

Working with Characters and `string` Objects

- To read a single character:
  - Use `cin`:

```
char ch;
cout << "Strike any key to continue";
cin >> ch;
```

Problem: will skip over blanks, tabs, <CR>

  - Use `cin.get()`:

```
cin.get(ch);
```

Will read the next character entered, even whitespace

Using `cin.get()` in Program 3-21

Program 3-21

```
1   // This program demonstrates three ways
2   // to use cin.get() to pause a program.
3   #include <iostream>
4   using namespace std;
5
6   int main()
7   {
8      char ch;
9
10     cout << "This program has paused. Press Enter to continue.";
11     cin.get(ch);
12     cout << "It has paused a second time. Please press Enter again.";
13     ch = cin.get();
14     cout << "It has paused a third time. Please press Enter again.";
15     cin.get();
16     cout << "Thank you!";
17     return 0;
18  }
```

Program Output with Example Input Shown in Bold

```
This program has paused. Press Enter to continue. [Enter]
It has paused a second time. Please press Enter again. [Enter]
It has paused a third time. Please press Enter again. [Enter]
Thank you!
```

Working with Characters and `string` Objects

- Mixing `cin >>` and `cin.get()` in the same program can cause input errors that are hard to detect
- To skip over unneeded characters that are still in the keyboard buffer, use `cin.ignore()`:

```
cin.ignore(); // skip next char
cin.ignore(10, '\n'); // skip the next
                      // 10 char. or until a '\n'
```

`string` Member Functions and Operators

- To find the length of a string:

```
string state = "Texas";
int size = state.length();
```

- To concatenate (join) multiple strings:

```
greeting2 = greeting1 + name1;
greeting1 = greeting1 + name2;
```

Or using the += combined assignment operator:

```
greeting1 += name2;
```

More Mathematical Library Functions

- Require `cmath` header file
- Take `double` as input, return a `double`
- Commonly used functions:

`sin`	Sine
`cos`	Cosine
`tan`	Tangent
`sqrt`	Square root
`log`	Natural (e) log
`abs`	Absolute value (takes and returns an int)

More Mathematical Library Functions

- These require `cstdlib` header file
- `rand()`: returns a random number (`int`) between `0` and the largest int the compute holds. Yields same sequence of numbers each time program is run.
- `srand(x)`: initializes random number generator with `unsigned int x`

Hand Tracing a Program

- Hand trace a program: act as if you are the computer, executing a program:
  - step through and 'execute' each statement, one-by-one
  - record the contents of variables after statement execution, using a hand trace chart (table)
- Useful to locate logic or mathematical errors

Program 3-27 with Hand Trace Chart

Program 3-27 **(with hand trace chart filled)**

```
1  // This program asks for three numbers, then
2  // displays the average of the numbers.
3  #include <iostream>
4  using namespace std;
5  int main()
6  {
7      double num1, num2, num3, avg;
8      cout << "Enter the first number: ";
9      cin >> num1;
10     cout << "Enter the second number: ";
11     cin >> num2;
12     cout << "Enter the third number: ";
13     cin >> num3;
14     avg = num1 + num2 + num3 / 3;
15     cout << "The average is " << avg << endl;
16     return 0;
17 }
```

num1	num2	num3	avg
?	?	?	?
?	?	?	?
10	?	?	?
10	?	?	?
10	20	?	?
10	20	?	?
10	20	30	?
10	20	30	40
10	20	30	40

Relational Operators

- Used to compare numbers to determine relative order
- Operators:

`>`	Greater than
`<`	Less than
`>=`	Greater than or equal to
`<=`	Less than or equal to
`==`	Equal to
`!=`	Not equal to

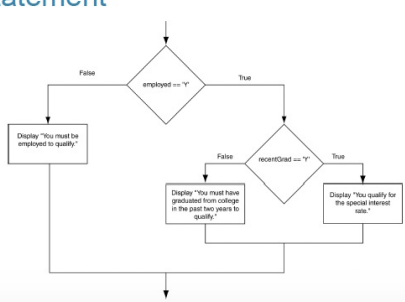
`if` Statement Notes

- Do not place `;` after `(`*`expression`*`)`
- Place *`statement`*`;` on a separate line after `(`*`expression`*`)`, indented:

```
if (score > 90)
    grade = 'A';
```

- Be careful testing `float`s and `double`s for equality
- `0` is `false`; any other value is `true`

Flowchart for a Nested `if` Statement



Flags

- Variable that signals a condition
- Usually implemented as a `bool` variable
- Can also be an integer
  - The value `0` is considered `false`
  - Any nonzero value is considered `true`
- As with other variables in functions, must be assigned an initial value before it is used

Logical Operators

- Used to create relational expressions from other relational expressions
- Operators, meaning, and explanation:

`&&`	AND	New relational expression is true if both expressions are true
`||`	OR	New relational expression is true if either expression is true
`!`	NOT	Reverses the value of an expression – true expression becomes false, and false becomes true

Logical Operator-Notes

- `!` has highest precedence, followed by `&&`, then `||`
- If the value of an expression can be determined by evaluating just the sub-expression on left side of a logical operator, then the sub-expression on the right side will not be evaluated (*short circuit evaluation*)

Menus

- Menu-driven program: program execution controlled by user selecting from a list of actions
- Menu: list of choices on the screen
- Menus can be implemented using `if/else if` statements

Menu-Driven Program Organization

- Display list of numbered or lettered choices for actions
- Prompt user to make selection
- Test user selection in *`expression`*
  - if a match, then execute code for action
  - if not, then go on to next *`expression`*

Validating User Input

- Input validation: inspecting input data to determine whether it is acceptable
- Bad output will be produced from bad input
- Can perform various tests:
  - Range
  - Reasonableness
  - Valid menu choice
  - Divide by zero

Input Validation in Program 4-19

```
16   int testScore;  // To hold a numeric test score
17
18   // Get the numeric test score.
19   cout << "Enter your numeric test score and I will\n"
20        << "tell you the letter grade you earned: ";
21   cin >> testScore;
22
23   // Validate the input and determine the grade.
24   if (testScore >= MIN_SCORE && testScore <= MAX_SCORE)
25   {
26      // Determine the letter grade.
27      if (testScore >= A_SCORE)
28        cout << "Your grade is A.\n";
29      else if (testScore >= B_SCORE)
30        cout << "Your grade is B.\n";
31      else if (testScore >= C_SCORE)
32        cout << "Your grade is C.\n";
33      else if (testScore >= D_SCORE)
34        cout << "Your grade is D.\n";
35      else
36        cout << "Your grade is F.\n";
37   }
38   else
39   {
40      // An invalid score was entered.
41      cout << "That is an invalid score. Run the program\n"
42           << "again and enter a value in the range of\n"
43           << MIN_SCORE << " through " << MAX_SCORE << ".\n";
44   }
```

Comparing Characters

- Characters are compared using their ASCII values
- 'A' < 'B'
  - The ASCII value of 'A' (65) is less than the ASCII value of 'B'(66)
- '1' < '2'
  - The ASCII value of '1' (49) is less than the ASCI value of '2' (50)
- Lowercase letters have higher ASCII codes than uppercase letters, so 'a' > 'Z'

Comparing `string` Objects

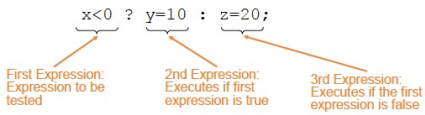
- Like characters, strings are compared using their ASCII values

```
string name1 = "Mary";
string name2 = "Mark";

name1 > name2   // true
name1 <= name2 // false
name1 != name2  // true

name1 < "Mary Jane" // true
```

The characters in each string must match before they are equal

The Conditional Operator

- Can use to create short `if/else` statements
- Format: `expr ? expr : expr;`



The Conditional Operator

- The value of a conditional expression is
  - The value of the second expression if the first expression is true
  - The value of the third expression if the first expression is false
- Parentheses `()` may be needed in an expression due to precedence of conditional operator

The `switch` Statement in Program 4-23

Program 4-23

```
// The switch statement in this program tells the user something
// he or she already knows: the data just entered!
#include <iostream>
using namespace std;

int main()
{
   char choice;

   cout << "Enter A, B, or C: ";
   cin >> choice;
   switch (choice)
   {
      case 'A': cout << "You entered A.\n";
                break;
      case 'B': cout << "You entered B.\n";
                break;
      case 'C': cout << "You entered C.\n";
                break;
      default:  cout << "You did not enter A, B, or C!\n";
   }
   return 0;
}
```

Program Output with Example Input Shown in Bold

```
Enter A, B, or C: B [Enter]
You entered B.
```

Program Output with Example Input Shown in Bold

```
Enter A, B, or C: F [Enter]
You did not enter A, B, or C!
```

`switch` Statement Requirements

1) *`expression`* must be an integer variable or an expression that evaluates to an integer value
2) *`exp1`* through *`expn`* must be constant integer expressions or literals, and must be unique in the `switch` statement
3) `default` is optional but recommended

`switch` Statement-How it Works

1) *`expression`* is evaluated
2) The value of *`expression`* is compared against *`exp1`* through *`expn`*.
3) If *`expression`* matches value *`expi`*, the program branches to the statement following *`expi`* and continues to the end of the `switch`
4) If no matching value is found, the program branches to the statement after `default:`

`break` Statement

- Used to exit a `switch` statement
- If it is left out, the program "falls through" the remaining statements in the `switch` statement

Using `switch` in Menu Systems

- `switch` statement is a natural choice for menu-driven program:
  - display the menu
  - then, get the user's menu selection
  - use user input as `expression` in `switch` statement
  - use menu choices as *`expr`* in `case` statements

More About Blocks and Scope

- Scope of a variable is the block in which it is defined, from the point of definition to the end of the block
- Usually defined at beginning of function
- May be defined close to first use

Inner Block Variable Definition in Program 4-29

```
16    if (income >= MIN_INCOME)
17    {
18       // Get the number of years at the current job.
19       cout << "How many years have you worked at "
20            << "your current job? ";
21       int years;     // Variable definition
22       cin >> years;
23
24       if (years > MIN_YEARS)
25          cout << "You qualify.\n";
26       else
27       {
28          cout << "You must have been employed for\n"
29               << "more than " << MIN_YEARS
30               << " years to qualify.\n";
31       }
32    }
```

Variables with the Same Name

- Variables defined inside { } have local or block scope
- When inside a block within another block, can define variables with the same name as in the outer block.
  - When in inner block, outer definition is not available
  - Not a good idea

Two Variables with the Same Name in Program 4-30

Program 4-30

```
1  // This program uses two variables with the name number.
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7     // Define a variable named number.
8     int number;
9
10    cout << "Enter a number greater than 0: ";
11    cin >> number;
12    if (number > 0)
13    {
14       int number;  // Another variable named number.
15       cout << "Now enter another number: ";
16       cin >> number;
17       cout << "The second number you entered was "
18            << number << endl;
19    }
20    cout << "Your first number was " << number << endl;
21    return 0;
22 }
```

Program Output with Example Input Shown in Bold

```
Enter a number greater than 0: 2 [Enter]
Now enter another number: 7 [Enter]
The second number you entered was 7
Your first number was 2
```